



File system, in simple language, can be explained as an interface (or rule) to store and access files on a storage device. Usually, in small embedded systems, file system is not used. Instead, the data is accessed directly using

Linux file system has a feature that simplifies GPIO access. In Linux, memory is separated into two spaces, i.e., kernel space and user space. Kernel space is part of memory occupied by the kernel modules, device drivers and everything that is under kernel's

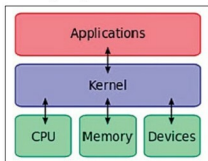


Fig. 1: Functional block diagram of Linux OS

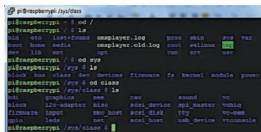


Fig. 2: Device models in class

```

git@raspberrypi:~$ cd /sys/class/gpio/
git@raspberrypi:/sys/class/gpio$ ls
export  gpiochip0  unexport
git@raspberrypi:/sys/class/gpio$ su
password:
root@raspberrypi:/sys/class/gpio# echo 4 > export
root@raspberrypi:/sys/class/gpio# ls
export  gpiochip0  unexport
root@raspberrypi:/sys/class/gpio# cd /dev
root@raspberrypi:/dev$ ls
active_low  direction  echo  power  subsystem  uevent  value

```

Fig. 3: Accessing the GPIO pin

Accessing GPIO using SYSfs interface

The process of accessing (setting or clearing) the GPIO pins of Raspberry Pi is given below:

Start at the terminal of Raspberry Pi, type 'cd /' and press the Enter key from keyboard to change directory to root. Now type 'ls' and press the Enter key to see contents of directory. Now you are in root file structure of Linux with lots of subdirectories. One of the directories is 'sys', which is nothing but SYSfs. SYSfs exports information about devices and drivers from the kernel space to user space. This mechanism is implemented as file system in 'sys' directory.

Now write the following and press
Enter key after each command:

```
$ cd sys
$ cd class
$ ls
```

You will see devices supported by the development board as shown in Fig. 2.

class gives a higher-level view (overview) of a device and hides implementation details (how internal logic is implemented). This *class* allows users to handle devices based on what they do, rather than how they work. In simple words, it is like a steering wheel of a car; driver just knows what it does and does not care how it is implemented inside the car.

Now write 'cd gpio' to enter the GPIO directory which contains two files, namely, export and unexport as shown in Fig. 3.

export. By writing GPIO number to this file, GPIO pin control is exported (or transferred) from kernel space to